

11 Dynamic Programming

In this chapter we will see efficient dynamic programming algorithms for all of the problems we discussed in Chapter 10. We start by motivating the concept of dynamic programming using Fibonacci numbers in Section 11.1. Then we convert our recursive algorithm for subset sum into a dynamic programming algorithm in Section 11.2. Having seen these two examples, we describe the general pattern for dynamic programming algorithms in Section 11.3, then give two more examples, longest increasing subsequence and longest common subsequence, in Sections 11.4 and 11.5.

11.1 Fibonacci Numbers

The Fibonacci numbers are defined by the recurrence:

$$\begin{aligned} F(1) &= 1 \\ F(2) &= 1 \\ F(x) &= F(x-1) + F(x-2) \quad \text{for } x \geq 3 \end{aligned}$$

We can give a very simple recursive algorithm for computing $F(x)$ using this recurrence:

FIB(x)
If $x \leq 2$:
 Return 1
Return FIB($x-1$) + FIB($x-2$)

We note that the runtime of FIB(x) is $\Omega(F(x))$, since calling FIB(x) results in $F(x)$ total recursive calls. The Fibonacci recurrence is not one that we can solve with the techniques that we have seen, but we can get a lower bound by noting that $F(x) = F(x-1) + F(x-2) \geq 2F(x-2)$. The recurrence $F(x) = 2F(x-2)$ solves to $F(x) \in \Theta(2^{x/2})$ (we can apply the same trick that we used in the proof of Theorem 53 by defining a new function by $F'(y) = F(\log_2 y)$ and using the master theorem on F'). Thus, $F(x) \in \Omega(2^{x/2})$, so the runtime of this algorithm is exponential in x (and doubly exponential in the length of the input).

Why is FIB so slow? If we look at the actual recursive calls that the algorithm makes, we see that

- FIB(x) calls FIB($x-1$) and FIB($x-2$)
- Our 1 call to FIB($x-1$) calls FIB($x-2$) and FIB($x-3$)
- Our 2 calls to FIB($x-2$) each call FIB($x-3$) and FIB($x-4$)
- Our 3 calls to FIB($x-3$) each call FIB($x-4$) and FIB($x-5$)
- And so on...

The problem is that our algorithm keeps making the *same* recursive calls over and over. We're making $\Omega(2^{x/2})$ recursive calls, even though the only inputs we're passing into the function are integers between 1 and x . We could make the algorithm much faster if we just computed FIB(i) once for each i from 1 to x .

The following algorithm exploits this idea:

FASTFIB(x)
If $x \leq 2$:
 Return 1
 $A \leftarrow$ an array of length x
 $A[1] \leftarrow 1$
 $A[2] \leftarrow 1$
For i from 3 to x
 $A[i] \leftarrow A[i-1] + A[i-2]$
Return $A[x]$

Note that while FIB was a recursive algorithm, FASTFIB is an iterative one. Instead of breaking the problem of computing $F(x)$ down into smaller problems, we start from the base cases of $F(1)$ and $F(2)$ and build up to $F(x)$, storing all of the intermediate values that we compute in an array. We call this process *memoization* and it is the tool that we use to convert any recursive algorithm into a dynamic programming one.

Note, however, that just because we are only computing x Fibonacci numbers, this does not mean that FASTFIB runs in $\Theta(x)$ time! As we saw in Chapter 9, we need to consider the size of these numbers. By our analysis above, $F(x) = \Omega(2^{x/2})$ so it is a number with $\Omega(x)$ digits, meaning that the additions inside the for loop take $\Omega(x)$ time in the worst case. In fact, we can show that the runtime of FASTFIB is $\Theta(x^2)$. It's also important to note that, while this runtime is exponentially faster than the runtime for FIB, it's still exponential in the length of the input, since the length of x is $\Theta(\log x)$ (We can't hope for a subexponential time algorithm since the length of $F(x)$ is exponential in the length of x).

11.2 Some Supplemental SecondSubsetSumSolver Speedups

In Section 10.1 we saw the following recurrence and corresponding exponential recursive algorithm for subset sum.

$$SS(A, k, i) = \begin{cases} True & \text{if } k = 0 \\ False & \text{if } i > n \text{ and } k > 0 \\ SS(A, k, i + 1) & \text{if } i \leq n, k > 0 \text{ and } A[i] > k \\ SS(A, k - A[i], i + 1) \vee SS(A, k, i + 1) & \text{otherwise} \end{cases}$$

```

SECONDSUBSETSUMSOLVER( $A[1, \dots, n], k, i = 1$ )
  If  $k = 0$ :
    Return True
  If  $i > n$ :
    Return False
  If  $A[i] > k$ :
    Return SECONDSUBSETSUMSOLVER( $A, k, i + 1$ )
  Return SECONDSUBSETSUMSOLVER( $A, k - A[i], i + 1$ )  $\vee$  SECONDSUBSETSUMSOLVER( $A, k, i + 1$ )

```

Let's try to speed up this algorithm by using the bottom-up iterative technique we saw in Section 11.1. This time our recurrence has 3 parameters instead of 1, so we can't easily store the answers to all relevant subproblems in a 1-dimensional array. We can note however that the array A is the same for all of our subproblems, so we can identify our subproblems just by the non-negative integers k and i . Since we have two non-negative integer parameters, a natural data structure to represent them is a 2-D array.

We can rewrite the recurrence in a way better suited for array indices as follows:

$$SS(i, j) = \begin{cases} True & \text{if } j = 0 \\ False & \text{if } i > n \text{ and } j > 0 \\ SS(i + 1, j) & \text{if } i \leq n, j > 0 \text{ and } A[i] > j \\ SS(i + 1, j - A[i]) \vee SS(i + 1, j) & \text{otherwise.} \end{cases}$$

Note that we have renamed k to j here to avoid confusion between the sums we are trying to get in our subproblems and the k which is part of the original input to the subset sum problem. We have also removed A as a parameter, since it is common to all subproblems, and we have rearranged the arguments to SS to be in alphabetical order.

While we typically assume that array indices start at 1 in this course, we don't need to rigidly hold to this convention when it comes at the expense of algorithmic clarity. The techniques we discussed for simulating arrays in the RAM model back in Section 5.4 can easily be extended to

arrays starting from 0, or even from a negative index. The important thing is to remember that an array needs space for all indices between its minimum index and max index, so if we have an array with indices ranging from a to b , it takes up space $\Theta(b - a)$ (assuming each element fits in a constant number of RAM model variables).

For this problem it is natural to let the first dimension of our DP array (corresponding to i) range from 1 to $n + 1$ (we need to allow indices larger than n since our base case is when $i > n$) and let the second dimension of the array (corresponding to j) range from 0 to k . We give the following pseudocode. Note that the values we assign to the entries of D are taken directly from our recurrence, simply replacing SS with D .

```

SPEEDIERSUBSETSUMSOLVER( $A[1, \dots, n], k$ ):
     $D[1, \dots, n + 1][0, \dots, k] \leftarrow$  an array of booleans
    For  $i$  from 1 to  $n + 1$ :
         $D[i, 0] \leftarrow \text{True}$ 
    For  $j$  from 1 to  $k$ :
         $D[n + 1, j] \leftarrow \text{False}$ 
    For  $i$  from  $n$  down to 1:
        For  $j$  from 1 to  $k$ :
            If  $A[i] > j$ :
                 $D[i, j] \leftarrow D[i + 1, j]$ 
            Else:
                 $D[i, j] \leftarrow D[i + 1, j - A[i]] \vee D[i + 1, j]$ 
    Return  $D[1, k]$ 

```

One part of this algorithm which may still seem mysterious or arbitrary is the order in which we chose to fill in the entries of D . Why did we start by filling in all the entries where $i = n + 1$ and then proceed to fill in the other entries in order of *decreasing* i value, rather than using a more normal loop structure where i goes from 1 to n ?

The answer comes from looking at our recurrence. In order to be able to fill in our DP array while iterating through the entries only once each, we need to ensure that whenever our algorithm is filling in $D[i, j]$ we have already filled in all the entries that $D[i, j]$ depends on, i.e. if $SS(i, j)$ calls $SS(i', j')$ we need to make sure that our algorithm fills in $D[i', j']$ before $D[i, j]$. Thus, we start by filling in the base cases, where $i = n + 1$, since these don't depend on any other entries. Then we fill in the other entries in order of decreasing value of i , because in every recursive call that SS makes, i increases. By going through the entries of D from largest i to smallest i we ensure that whenever we are computing $D[i, j]$, the entries that it depends on, namely $D[i + 1, j]$ and possibly $D[i + 1, j - A[i]]$, are already filled in.

Theorem 54. SPEEDIERSUBSETSUMSOLVER runs in time $O(nk)$ and uses $O(nk)$ space.

Proof. Creating an array of size $O(nk)$ on the first line takes $O(nk)$ time. Filling in the base cases takes time $O(n + k)$. The nested for loops iterate through $O(nk)$ entries of D , and fill in in each one in $O(1)$ time since we are only performing a constant number of operations for each (the comparison $A[i] > j$ takes time $O(1)$ since we know $0 \leq j \leq k$). Thus, the overall runtime is $O(nk)$. Storing the array D takes $O(nk)$ space, and all other variables in the algorithm take constant space, so the space usage is $O(nk)$. $\square$

It's important to realize that dynamic programming algorithms do not always run in polynomial time. Dynamic programming allows us to design algorithms whose runtime depends on the number of different subproblems rather than on how many recursive calls those subproblems make, but the degree to which this speeds up the algorithm depends on how many recursive calls were using the exact same inputs. If there are exponentially many different inputs to our recursive function, then our DP table will have exponential size and filling it in will take exponential time.

In particular, note that the runtime bound of $O(nk)$ that we proved in Theorem 54 is actually still exponential in the input length. If k is a number with n digits then the value of k is about 10^n and our runtime bound becomes $O(n10^n)$. If k is small, however this algorithm is efficient, and even if k is large this algorithm is still much faster than the recursive backtracking version we developed last chapter.

11.3 Designing Dynamic Programming Algorithms

Let's describe and generalize the process that we saw in the previous two sections. How do we develop a dynamic programming algorithm?

The starting point of any dynamic programming algorithm is a recurrence. We can view dynamic programming as simply smart recursion. The challenging part of coming up with a dynamic programming algorithm is coming up with the right recurrence. This is why we spent Chapter 10 talking about designing recursive algorithms. Once we figure out the right recurrence, the steps to create the DP algorithm are pretty mechanical.

1. Formulate the problem recursively
 - (a) Describe in English what your subproblems are (i.e. what problem your recurrence is solving)
 - Often this will require introducing additional parameters. Clearly explain what each one means.
 - (b) Give a recurrence or a recursive algorithm to solve your subproblems.
 - Your recurrence should solve the problem using smaller versions of the exact same problem.
 - Make sure to include the base case.
 - (c) Describe how to use your recurrence to solve the original problem (e.g. what should the default values be for your extra parameters?).
2. Give an algorithm which builds solutions to your recurrence from the bottom up
 - (a) Identify what subproblems you need to solve (what are the possible inputs to your recurrence?)
 - (b) Choose an appropriate memoization data structure (e.g. a multidimensional array)
 - (c) Figure out the evaluation order (make sure that when you are filling in an entry of your DP array, you have already filled in the entries it depends on)
 - (d) Analyze the runtime (usually just (# of subproblems) · (work per subproblem))

When you are asked to give an algorithm for a problem and want to use dynamic programming, you should make sure to define your subproblems in English (step 1(a)) so that it's clear what you're trying to do, whether or not you intend to give full pseudocode for your algorithm.

Then you should either explain the remaining steps (1(b), 1(c), and 2(a)-(d)) in English, at which point you have fully defined an algorithm, or you should give iterative pseudocode for your algorithm, which implicitly answers all the questions from these steps. In this chapter we will both explain the choices we are making for the algorithm in English and give the iterative pseudocode in order to maximize clarity.

11.4 Longest Increasing Subsequence Revisited

For another example of dynamic programming, let's revisit the longest increasing subsequence problem which we looked at in Section 10.2. Recall that we are given an array A of integers, and the goal is to find the length of the longest subsequence which is strictly increasing (meaning each element is greater than the previous one).

Recall that we defined $LIS(i, j)$ as the length of the longest increasing subsequence of $A[i, \dots, n]$ where every element is larger than $A[j]$ and gave the following recurrence:

$$LIS(i, j) = \begin{cases} 0 & \text{if } i > n \\ LIS(i + 1, j) & \text{if } i \leq n \text{ and } A[i] \leq A[j] \\ \max\{LIS(i + 1, j), 1 + LIS(i + 1, i)\} & \text{otherwise} \end{cases}$$

We can memoize the function LIS into a 2D array $D[1, \dots, n+1][1, \dots, n]$. Each entry $D[i, j]$ depends only on entries in the next row $D[i+1, \cdot]$, so we can fill the array from bottom to top (we arbitrarily choose to fill each row from left to right).

```

LIS-SOLVER( $A[1, \dots, n]$ ):
   $D \leftarrow$  a 2D array of size  $(n+1) \times n$ 
  For  $j$  from 1 to  $n$  ▷ Base cases
     $D[n+1, j] \leftarrow 0$ 

  For  $i$  from  $n$  down to 1
    For  $j$  from 1 to  $n$ 
      If  $A[i] \leq A[j]$ 
         $D[i, j] \leftarrow D[i+1, j]$ 
      Else
         $D[i, j] \leftarrow \max\{D[i+1, j], 1 + D[i+1, i]\}$ 

  Return  $\max_{1 \leq i \leq n} (1 + D[i+1, i])$ 

```

Theorem 55. $LIS-SOLVER(A)$ returns the length of the longest increasing subsequence of A , runs in time $\Theta(n^2)$ and uses $\Theta(n^2)$ space.

Proof. The correctness of this algorithm follows from the correctness of the recurrence: Here our recurrence represents the choice of whether or not to include $A[i]$ in our subsequence. If $A[i] \leq A[j]$, we have no choice, we have to skip $A[i]$. If $A[i] > A[j]$, we can include it, in which case we increase the length of our sequence by 1, but change our lower bound on future elements to $A[i]$, or we can skip it. We choose whichever option gives a longer subsequence. To solve the original problem, we compute $\max_{1 \leq i \leq n} (1 + D[i+1, i])$ because this is the max, over all choices of first element, of the longest increasing subsequence we can get starting with that element.

Now we analyze the runtime. Our array has $\Theta(n^2)$ entries, and we can compute each one in $\Theta(1)$ time (computing a new entry requires comparing $A[i]$ and $A[j]$ and then either copying a value we already computed or taking the max of two values we already computed and copying one of them, all of which are constant time operations). Therefore the total runtime to fill in our table is $\Theta(n^2) \cdot \Theta(1) = \Theta(n^2)$. Computing the final answer from our table requires taking the max over n entries which takes $\Theta(n)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time.

Finally, we note that since our DP table has $\Theta(n^2)$ entries, the space complexity is $\Theta(n^2)$. □

If we want to be clever, we can reduce the space complexity of our DP algorithm by defining our subproblems and recurrence slightly differently.

Let $LIS(i)$ denote the length of the longest increasing subsequence of $A[i, \dots, n]$ that begins with $A[i]$. We give the following recurrence:

$$LIS(i) = 1 + \max_{i < j \leq n \text{ with } A[j] > A[i]} LIS(j).$$

Here our recurrence represents choosing the next element in the sequence after $A[i]$. We define $\max \emptyset = 0$ so that this recurrence covers the base cases. To solve the original problem, we need to compute $\max_{1 \leq i \leq n} LIS(i)$.

We can memoize the function LIS into a 1D array $D[1, \dots, n]$. Each entry $D[i]$ depends only on subsequent entries, so we can fill the array in order of decreasing values of i .

```

LIS-SOLVER2( $A[1, \dots, n]$ ):
   $D \leftarrow$  a 1D array of size  $n$ 
  For  $i$  from  $n$  down to 1
     $max\_len \leftarrow 0$ 
    For  $j$  from  $i+1$  to  $n$ 
      If  $A[j] > A[i]$  and  $D[j] > max\_len$ 
         $max\_len \leftarrow D[j]$ 
     $D[i] \leftarrow 1 + max\_len$ 

  Return  $\max_{1 \leq i \leq n} D[i]$ 

```

Theorem 56. LIS-SOLVER2(A) returns the length of the longest increasing subsequence of A , runs in time $\Theta(n^2)$ and uses $\Theta(n)$ space.

Proof. Once again, our algorithm is simply implementing our recurrence, so the correctness follows from the correctness of the recurrence. If we have an increasing subsequence starting with $A[i]$ then the valid choices for the next element are all elements after index i which are greater than $A[i]$, and we pick whichever one gives us the longest overall subsequence. The total length of our subsequence is 1 (for $A[i]$) plus the length of the subsequence starting with the next element.

Our array has $\Theta(n)$ entries now, but each one takes $\Theta(n)$ time to compute, since computing $D[i]$ requires looping over all of $A[i + 1, \dots, n]$. Therefore the total runtime to fill in our table is $\Theta(n) \cdot \Theta(n) = \Theta(n^2)$. Computing the final answer from our table requires taking the max over n entries which takes $\Theta(n)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time. Since we are only using a 1D array this time, however, the space complexity is only $\Theta(n)$. $\square$

This example illustrates that there can be multiple valid choices for how we define subproblems, and they are not always equivalent. In this example, our second version of the subproblems gave the same runtime but better space complexity. In general it's possible that some subproblem definitions may yield faster runtimes than others. When developing your recurrence its worth taking time to think whether there may be multiple options for how to define the subproblems and which one will yield the smallest DP array.

11.5 Longest Common Subsequence Revisited

Now let's revisit the last problem we saw a recursive backtracking algorithm for last chapter: longest common subsequence. Recall that in this problem we are given two arrays $A[1, \dots, m]$ and $B[1, \dots, n]$ and we want to find the longest sequence which is a subsequence of both A and B .

Recall that in Section 10.4 we defined $LCS(i, j)$ as the length of the longest common subsequence of $A[i : m]$ and $B[j : n]$. We defined an auxiliary function I by

$$I(i, j) := \{(i', j') \mid i \leq i' \leq m, j \leq j' \leq n \text{ and } A[i'] = B[j']\}.$$

That is, $I(i, j)$ is the set of all pairs of indices (i', j') in our range of valid indices where $A[i'] = B[j']$. Then we gave the following recurrence for LCS :

$$LCS(i, j) = \begin{cases} 0 & \text{if } I(i, j) = \emptyset \\ \max_{(i', j') \in I(i, j)} 1 + LCS(i' + 1, j' + 1) & \text{otherwise} \end{cases}.$$

We noted that the solution to the original problem is $LCS(1, 1)$.

Since this recurrence has two parameters, we can memoize it with a 2-D array $D[1, \dots, m + 1][1, \dots, n + 1]$. Since each entry in the array depends only on entries with larger values of i and j , we fill in the array in order of decreasing values of i .

We summarize these choices with the following pseudocode:

```

LCS-SOLVER( $A[1, \dots, m], B[1, \dots, n]$ ):
   $D[1, \dots, m + 1][1, \dots, n + 1] \leftarrow$  a 2-D array of integers
  For  $i$  from  $m + 1$  down to 1
    For  $j$  from 1 to  $n + 1$ 
       $max\_len \leftarrow 0$ 
      For  $i'$  from  $i$  to  $m$ :
        For  $j'$  from  $j$  to  $n$ :
          If  $A[i'] = B[j']$ 
             $max\_len \leftarrow \max(max\_len, 1 + D[i' + 1, j' + 1])$ 
       $D[i, j] \leftarrow max\_len$ 
  Return  $D[1, 1]$ 

```

Theorem 57. LCS-SOLVER returns the length of the longest common subsequence of A and B , runs in time $\Theta(m^2n^2)$ and uses $\Theta(mn)$ space.

Proof. Correctness: Every pair of a character from $A[i, \dots, m]$ and a matching character from $B[j, \dots, n]$ (not necessarily at the same index) forms a potential starting point for a common subsequence of these two arrays. The innermost two loops in our algorithm implement the “max” in our recurrence by considering all possible pairs of starting characters. Then, for each pair of matching characters, we determine the longest common subsequence we could obtain starting with those characters, which is 1 (for the first character we’ve chosen) plus the longest common subsequence of the remainder of the arrays.

Runtime: We have $\Theta(mn)$ entries in our DP array, and we spend $\Theta((m-i)(n-j))$ time filling in the entry $D[i, j]$ (since we need to iterate over all pairs (i', j') with $i \leq i' \leq m$ and $j \leq j' \leq n$). This is $O(mn)$ for all entries and $\Omega(mn)$ for at least a constant fraction of the entries (e.g. the ones where $i \leq m/2$ and $j \leq n/2$). Thus the overall runtime is $\Theta(mn) \cdot \Theta(mn) = \Theta(m^2n^2)$.

Space: We use an $m \times n$ array (plus a constant number of integer variables), so the algorithm takes $\Theta(mn)$ space. $\square$

We’ve shown that LCS-SOLVER is a polynomial time algorithm, which is a big improvement over the exponential time algorithm we gave in Section 10.4. Nevertheless, we can still try to do better.

In Section 11.4 we were able to improve the space complexity of our algorithm by giving a recurrence with fewer parameters, allowing us to use a smaller DP array (at the cost of taking longer to compute each entry).

For the longest common subsequence problem, having our recurrence take in an index for each array seems fairly necessary, but we can design a recurrence that will let us compute the entries of the DP array more efficiently. Since looking ahead at all the possible choices for the next element of the common subsequence isn’t allowing us to use a smaller DP table here, it makes sense to try a perspective like the one we took for our first algorithm for longest increasing subsequence and view the choice we are making at each step of the recurrence as “Should we include or skip the element at the current index” rather than “Which element should we add to our common subsequence next?”.

In particular, if the first elements of A and B are the same, we might as well use that element as the first element of a common subsequence (Any common subsequence of $A[i, \dots, m]$ and $B[j, \dots, n]$ that didn’t start with $A[i]$ could be made longer by prepending $A[i]$). Otherwise, we know that our common subsequence can’t use both the first element of A and the first element of B , so we can pick one of these two to discard and recurse on the subproblem where one of the arrays is one element shorter. Since we don’t know which array we should discard the first element from, we use our standard recursive backtracking approach of trying both and taking the max.

Once again we’ll define $LCS(i, j)$ as the length of the longest common subsequence of $A[i : m]$ and $B[j : n]$. This time we give the following alternative recurrence:

$$LCS(i, j) = \begin{cases} 0 & \text{if } i > m \text{ or } j > n \\ 1 + LCS(i + 1, j + 1) & \text{if } A[i] = B[j] \\ \max(LCS(i + 1, j), LCS(i, j + 1)) & \text{otherwise} \end{cases}.$$

We can memoize this recurrence into an $m + 1 \times n + 1$ array. This time, our recurrence has recursive calls where i stays the same, so just evaluating our subproblems in order of decreasing value of i is not enough to ensure that when we are computing $D[i, j]$ we have already computed the entries it depends on. We also need to make sure that, for a given value of i , we iterate through the values of j from largest to smallest (because our base case is when $j > n$ and j increases when we recurse keeping i the same).

This gives us the following DP pseudocode:

```

LCS-SOLVER2( $A[1, \dots, m], B[1, \dots, n]$ ):
   $D[1, \dots, m+1][1, \dots, n+1] \leftarrow$  a 2-D array of integers
  For  $i$  from  $m+1$  down to 1
    For  $j$  from  $n+1$  down to 1
      If  $i > m$  or  $j > n$ :
         $D[i, j] \leftarrow 0$ 
      Else If  $A[i] = B[j]$ :
         $D[i, j] \leftarrow 1 + D[i+1, j+1]$ 
      Else:
         $D[i, j] \leftarrow \max(D[i+1, j], D[i, j+1])$ 
  Return  $D[1, 1]$ 

```

Once again we have $\Theta(mn)$ subproblems, but this time it takes only $\Theta(1)$ work to solve each of them, so our overall runtime is $\Theta(mn)$

11.6 Comprehension Questions

- Suppose that we are creating a dynamic programming algorithm for solving a problem about an array A and want to memoize the following recurrence into an array $D[1, \dots, n][1, \dots, n]$

$$f(i, j) = \begin{cases} 0 & \text{if } i \geq j \\ f(i+1, j) & \text{if } i < j \text{ and } A[i] \geq A[j] \\ \max\{f(i+1, j), 1 + f(i, j-1)\} & \text{otherwise} \end{cases}$$

Which of the following evaluation orders would be appropriate for filling in our DP table? (select all that apply)

- Fill in the entries from smallest value of i to largest. For entries with the same value of i , fill them in from smallest value of j to largest.
 - Fill in the entries from smallest value of i to largest. For entries with the same value of i , fill them in from largest value of j to smallest.
 - Fill in the entries from largest value of i to smallest. For entries with the same value of i , fill them in from smallest value of j to largest.
 - Fill in the entries from largest value of i to smallest. For entries with the same value of i , fill them in from largest value of j to smallest.
- What is the runtime of FASTFIB(x) in terms of x (not the input length n)?
 - In this chapter we designed a DP algorithm SPEEDIERSUBSETSUMSOLVER for subset sum. What is the runtime of SPEEDIERSUBSETSUMSOLVER($A[1, \dots, n], n^3$), i.e. what is the running time of the algorithm if we require k to be n^3 ? Express your answer as a tight big- O bound in terms of n .
 - In this chapter we designed a DP algorithm LIS-SOLVER2 based on the recurrence

$$LIS(i) = 1 + \max_{i < j \leq n \text{ with } A[j] > A[i]} LIS(j).$$

What is the space complexity of LIS-SOLVER2?

- Suppose that we are trying to solve a problem where the input is an array $A[1, \dots, n]$. We have come up with a recurrence for the problem and memoized it into an $n \times n$ array which our DP algorithm will fill in. What can we say about the runtime of this DP algorithm?
 - The runtime of the algorithm is $O(n^2)$
 - The runtime of the algorithm is $\Theta(n^2)$
 - The runtime of the algorithm is $\Omega(n^2)$
 - This is not enough information to say anything about the algorithm's runtime.

11.7 Exercises

1. We define a sequence of integers $x_1, x_2, \dots, x_n$ as *converging* if for every $i \geq 3$, x_i lies in-between x_{i-1} and x_{i-2} (i.e. either $x_{i-2} < x_i < x_{i-1}$ or $x_{i-1} < x_i < x_{i-2}$). For example, 1, 100, 40, 60, 47, 59, 50, 52 is a converging sequence. Give a polynomial time algorithm which takes as input a sequence $a_1, \dots, a_n$ of n distinct positive integers, represented by an array $A[1, \dots, n]$ and returns the length of a longest converging subsequence of $a_1, \dots, a_n$. For example, if

$$A = [101, 1, 40, 100, 90, 80, 30, 50, 43, 87, 48, 46, 44, 47, 103, 102]$$

then your algorithm should return 8 because a longest converging subsequence of A is 1, 100, 30, 50, 43, 48, 44, 47, which has length 8.

2. You are planning a road trip between cities A and B . Fortunately, they are connected by a long straight highway. Unfortunately, you are going to have to stop for fuel several times, and all of the gas stations on this highway use a very unusual pricing system. Gas is sold by the tank, meaning that you pay the same fixed amount to refill your tank, whether that takes one gallon or twenty.

Suppose that you are given d , the distance in miles that your car can travel on a full tank of gas, as well as two arrays $G[1, \dots, n]$ and $P[1, \dots, n]$, where $G[i]$ is the location of the i th gas station, given as a distance in miles from your starting point, and $P[i]$ is the price (in dollars) to fill your tank at the i th gas station. Assume that your trip begins at the first gas station (so $G[1] = 0$) with an empty tank of gas, and ends at the n th gas station (where you do not need to refuel). Assume that it is possible to complete the trip (i.e. there is no stretch of d miles with no gas stations). Give an efficient algorithm to calculate the minimum amount that you can pay for gas while making this trip.

3. Susan is going door to door in her neighborhood selling boxes of cookies. Susan's cookies come in two different kinds. She has x boxes of chocolate chip cookies and y boxes of oatmeal raisin cookies. Susan's neighborhood consists of n houses arranged in a straight line. Each household is willing to buy at most one box of cookies, and they each value the two kinds of cookies differently. Fortunately, Susan has done extensive market research, and has compiled two arrays, $A[1, \dots, n]$ and $B[1, \dots, n]$, where $A[i]$ is the price (in cents) that the i th house will pay for a box of chocolate chip cookies, $B[i]$ is the price that they will pay for a box of oatmeal raisin cookies (You may assume that all of these prices are non-negative integers).

Susan needs to decide which of her two kinds of cookies to offer at each house in order to maximize the revenue she earns from her limited cookie stash. Give an efficient algorithm to help her with this problem. Your algorithm should take as input the values x and y and the arrays A and B . Your algorithm should output a length n array where each entry is either "chocolate chip", "oatmeal raisin", or "nothing", and for each i the i th entry describes which type of cookie Susan should sell at the i th house in order to maximize the total amount she earns from selling these cookies.

4. Suppose you are given a list of n positive integers separated by plus and minus signs, for example

$$5 + 2 - 3 + 7 - 4 - 12$$

You can change the value of this expression by adding parentheses in different positions. For example:

$$\begin{aligned} 5 + 2 - 3 + 7 - 4 - 12 &= -5 \\ 5 + 2 - (3 + 7) - (4 - 12) &= 5 \\ 5 + (2 - 3 + 7 - (4 - 12)) &= 19 \end{aligned}$$

Give an efficient algorithm to find a placement of parentheses which maximizes the value of the expression. You may only use the parentheses to group additions and subtractions, you are not allowed to create implicit multiplication (e.g. parenthesizing $5 - 2$ as $5(-2)$ is not allowed).

5. We define a *ones decomposition* as a valid arithmetic expression that can be created using only the 5 symbols 1, +, ×, (, and). Each ones decomposition represents some positive integer, and most positive integers have multiple ones decompositions. For example, the number 14 can be represented by any of the following ones decompositions

$$\begin{aligned}
 &1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 \\
 &((1 + 1 + 1) \times (1 + 1 + 1)) + ((1 + 1) \times (1 + 1)) + 1 \\
 &\quad (1 + 1) \times (1 + 1 + 1 + 1 + 1 + 1 + 1) \\
 &(1 + 1) \times (((1 + 1) \times (1 + 1 + 1)) + 1)
 \end{aligned}$$

Give an algorithm which takes as input a positive integer x and outputs the minimum number of 1s in a ones decomposition for x . For example, given the input 14, your algorithm should return 8 (the last ones decomposition given above uses a minimum number of 1s). The runtime of your algorithm should be polynomial in x .

6. We define a balanced string of braces recursively as follows:

- ε (the empty string) is a balanced string of braces.
- If s is a balanced string of braces then so is $\{s\}$.
- If s is a balanced string of braces then so is $\langle s \rangle$.
- If s and t are balanced strings of braces, then so is st (the concatenation of s and t).

For example, the string $\{\langle \rangle\}\langle \rangle$ is a balanced string of braces, but $\{\langle \rangle\}$ is not.

Suppose that you are given an array $A[1, \dots, n]$ where each element of A is one of $\langle$, $\rangle$, $\{$, or $\}$.

- Give an $O(n)$ -time algorithm to determine whether or not A represents a balanced string of braces.
 - Give an $O(n^3)$ -time algorithm to find the length of the longest subsequence of A which is a balanced string of braces.
7. Linear Town is a town consisting of a single very long straight road running east to west. In the middle of this street is a pizza place where you have been hired as a pizza delivery driver. When you show up for your first day of work, you're informed that a driver quit in the middle of the previous shift, leaving you with a stack of n pizzas that urgently need to be delivered.

The customers who ordered these pizzas are furious at how long they have had to wait and have started using fake accounts to leave 1 star reviews of your restaurant. For every minute that a customer goes without receiving their pizza, they will leave another 1 star review. Your goal is to deliver all the pizzas while incurring as few negative reviews as possible.

Fortunately, there's no traffic right now, so you will be able to drive at a consistent speed while delivering the pizzas. Since the entire city is one long street, instead of needing to keep track of a list of addresses, the location of a house can be expressed just by how many minutes it takes you to drive there from the restaurant and whether it is west or east of the restaurant.

You are given a sorted array $A[1, \dots, n]$ of integers, where $A[i]$ is the number of minutes you need to drive east from the restaurant to reach the house (negative numbers correspond to houses which are west of the restaurant). Give an $O(n^2)$ -time algorithm to determine the minimum number of 1-star reviews that you can incur in the process of delivering the pizzas.

For example, if $A = [-10, -9, -8, 1, 7]$, then the optimal route is $[1, -8, -9, -10, 7]$ which incurs 63 1-star reviews (5 while driving to 1, 36 more while driving to -8, 3 more while driving to -9, 2 more while driving to -10, and 17 more while driving to 7).